



Note

Computing the longest topological common subsequence of a symbol-wise totally ordered directed acyclic graph and a sequence

Yoshifumi Sakai*

Graduate School of Agricultural Science, Tohoku University, Sendai 981-8555, Japan

ARTICLE INFO

Article history:

Received 16 October 2007

Received in revised form 5 December 2008

Accepted 20 March 2009

Communicated by A. Apostolico

Keywords:

Algorithms

Longest common subsequence

Topological sort

ABSTRACT

Let G be a directed acyclic graph, each vertex of which is labeled with a symbol, and having, for any such symbol, a path in which all of the vertices labeled with the symbol appear with vertices labeled with other symbols. Let B be a sequence of symbols. This article proposes a polynomial-time algorithm for computing one of the longest possible common subsequences of a sequence specified by any topological sort of G and the sequence B .

© 2009 Elsevier B.V. All rights reserved.

1. Introduction

The longest common subsequence problem for two sequences consists of finding a longest subsequence common to the two sequences, where a subsequence of a sequence A is a sequence obtained from A by deleting zero or more symbols at any position [3]. Significant effort has been devoted to developing efficient algorithms for solving this classic problem [7,8,1,4,2,5,12], because it has several important applications in computer science and molecular biology. For example, in molecular biology, the length of the longest common subsequence is used as one measure of the similarity of two given biological sequences [6]. This article considers a generalization of the problem, where one of the two sequences is replaced by a set of sequences specified by a given directed acyclic graph G , each vertex of which is labeled with a symbol. We propose a polynomial-time algorithm for finding one of the longest possible common subsequences of any sequence in the set with respect to G and a sequence B . One such problem, where the set consists of all sequences specified by directed paths on G , was investigated in [11,6]. This problem arises in testing the similarity between a DNA sequence and an amino acid sequence.

In this article, we consider the set consisting of all sequences specified by topological sorts of G , where G is a given directed acyclic graph that has, for any symbol, a path in which all vertices labeled with the symbol appear, possibly along with vertices labeled with other symbols. Such a graph will be called symbol-wise totally ordered. The problem considered in the present article may arise, for example, in aligning a sequence with an irredundant multiple alignment of sequences with gap flexibility being taken into account. Let S be a set of sequences, and let A be a multiple alignment of S , each column of which contains elements labeled with the same symbol, except for gaps (which may be obtained from a general alignment of S by separating each column containing elements labeled with more than one symbol into columns satisfying the restriction on A). Consider the situation in which the positions of inserted gaps in A are not important, and hence A and any multiple alignment of S obtained by changing the order of columns of A may be considered to be equivalent. For example, if $S = \{aacc, bacac, abbbc\}$, then the two multiple alignments of length 8 of S displayed in Fig. 1(a) and (b) can be considered to be identical, because alignment (b) is obtained by listing the i th columns of alignment (a) in the order

* Tel.: +81 22 717 8838.

E-mail address: sakai@biochem.tohoku.ac.jp.

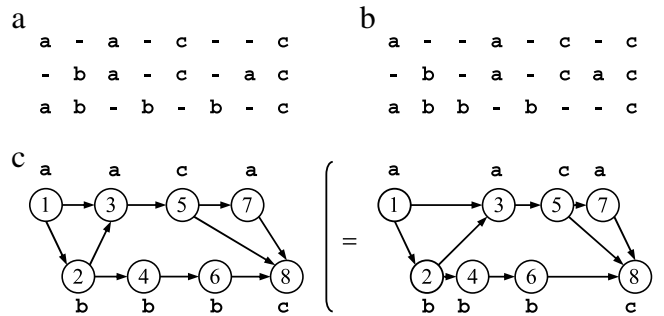


Fig. 1. (a) An alignment of sequences aacc, bacac, and abbbc. (b) Another alignment of the same sequences, which is obtained by listing the i th columns of the alignment (a) in the order of 1, 2, 4, 3, 6, 5, 7, 8. (c) The trace layout graph representing equivalent alignments including both (a) and (b).

of 1, 2, 4, 3, 6, 5, 7, 8. Note that not all changes of the order of columns of A yield alignments of S , but only those that do not violate the order of symbols in every sequence in S . Although, in general, there exists an exponential number of such equivalent multiple alignments of S , each alignment corresponds to a specific topological sort of the trace layout graph [9] for A . The trace layout graph for the above example is depicted in Fig. 1(c), where each vertex i corresponds to the i th column of alignment (a) and is labeled with the specific symbol appearing in the column. It can be easily verified that A is irredundant in the sense that there does not exist a pair of columns of A such that merging the two columns would provide a shorter alignment of S , if and only if the trace layout graph for A is symbol-wise totally ordered. Thus, the problem of computing one of the shortest multiple alignments by aligning an irredundant multiple alignment A of S with an additional sequence B is essentially equivalent to the problem of finding one of the longest possible common subsequences of any sequence arising from a topological sort of the trace layout graph for A and B .

In the present article, we propose an $O(|G|^2|B|^2)$ -time algorithm for computing one of the longest possible common subsequences of any sequence in the set of all sequences specified by topological sorts of a symbol-wise totally ordered dag G and a sequence B , where $|G|$ denotes the number of vertices of G and $|B|$ denotes the length of B .

2. Preliminaries

Let Σ be a finite alphabet set. For any sequence A over Σ , let $|A|$ denote the length of A . For any index i , $1 \leq i \leq |A|$, let $A[i]$ denote the i th element of A , and let A_i denote the prefix of A of length i . A sequence A' is a subsequence of A , if there exist $|A'|$ indices $i_1, i_2, \dots, i_{|A'|}$, $1 \leq i_1 < i_2 < \dots < i_{|A'|} \leq |A|$, such that $A'[p] = A[i_p]$ for any index p , $1 \leq p \leq |A'|$. For any directed acyclic graph (dag) G , let $|G|$ denote the number of vertices of G . Assume that each vertex of any dag is labeled with a symbol in Σ . For any vertex v of G , let $G[v]$ denote the symbol with which v is labeled.

A topological sort of a dag is a sequence consisting of all vertices of the dag such that any vertex appears exactly once, and u occurs before v if the dag has an edge from u to v . Any topological sort of a dag naturally specifies a sequence over Σ having a length equal to the number of vertices of the dag, the element of which at any position i is the symbol with which the i th vertex of the topological sort is labeled. For two dags G and H , any common subsequence of the sequence A specified by some topological sort of G and the sequence B specified by some topological sort of H is called a *topological common subsequence* of (G, H) .

A dag is called *symbol-wise totally ordered* if, for any symbol in Σ , there exists a path of the dag in which all vertices labeled with the symbol appear, possibly along with vertices labeled with other symbols. Note that, any sequence B over Σ can be thought of as a symbol-wise totally ordered dag, because the dag consisting of vertices v_i labeled with $B[i]$ for all indices i , $1 \leq i \leq |B|$, and edges from v_i to v_{i+1} for all indices i , $1 \leq i \leq |B| - 1$, is symbol-wise totally ordered, and has exactly one topological sort, by which B is specified. This article considers the problem of finding one of the longest topological common subsequences of a symbol-wise totally ordered dag G and a sequence B .

3. Matching pairs

Let G be a symbol-wise totally ordered dag, and let B be a sequence. The problem of finding one of the longest topological common subsequences of (G, B) is equivalent to the problem of finding one of the maximal independent sets of a graph determined from G and B . Since the idea underlying the proposed algorithm is explained in terms of this maximal independent set problem, rather than in terms of the longest topological common subsequence problem, let us first consider the maximal independent set problem before presenting the algorithm.

Let us refer to a pair (v, j) of any vertex v of G and any index j , $1 \leq j \leq |B|$, as a *matching pair* of (G, B) if $G[v] = B[j]$ holds. There exists an l -length topological common subsequence of (G, B) if and only if there exist a topological sort T of G , l indices $i_1, i_2, \dots, i_l$, $1 \leq i_1 < i_2 < \dots < i_l \leq |G|$, and l indices $j_1, j_2, \dots, j_l$, $1 \leq j_1 < j_2 < \dots < j_l \leq |B|$, such that $G[T[i_p]] = B[j_p]$ for any index p , $1 \leq p \leq l$, where $T[i_p]$ denotes the i_p th vertex of T . In such a case, we say that the set consisting of matching pairs $(T[i_1], j_1), (T[i_2], j_2), \dots, (T[i_l], j_l)$ represents a topological common subsequence of (G, B) .

For any vertices u and v of G , let $u \rightsquigarrow v$ indicate that either G has a path from u to v or $u = v$. Let $u \not\rightsquigarrow v$ denote that $u \rightsquigarrow v$ does not hold. For any matching pairs $y = (u, j)$ and $z = (v, k)$ of (G, B) , let y precede z if both $u \rightsquigarrow v$ and $j \geq k$ hold but $y \neq z$. Note that, for any matching pairs x, y , and z , if x precedes y and y precedes z , then x precedes z . Let $y < z$ denote that y precedes z , and let $y \leq z$ denote that either $y < z$ or $y = z$. In addition, let $y \not< z$ denote that y does not precede z , and let $y \not\leq z$ denote that neither $y < z$ nor $y = z$. A set of matching pairs is *independent* if y does not precede z for any matching pairs y and z in the set. Let an independent set be maximal if the size of that set is maximal among all independent sets.

The following lemma tells us that the problem of finding one of the longest topological common subsequences of (G, B) is equivalent to the problem of finding one of the maximal independent sets of matching pairs of (G, B) .

Lemma 1. For any set Z of matching pairs of (G, B) , Z represents a topological common subsequence of (G, B) if and only if Z is independent.

Proof. Let Z consist of l matching pairs $(v_1, j_1), (v_2, j_2), \dots, (v_l, j_l)$ of (G, B) such that $j_1 \leq j_2 \leq \dots \leq j_l$.

Assume that Z represents a topological common subsequence of (G, B) . It follows that $j_1 < j_2 < \dots < j_l$, and that there exists a topological sort T of G and l indices $i_1, i_2, \dots, i_l$, $1 \leq i_1 < i_2 < \dots < i_l \leq |G|$, such that $T[i_p] = v_p$ for any index p , $1 \leq p \leq l$. For any indices p and q , $1 \leq p < q \leq l$, it follows from $j_p < j_q$ that (v_p, j_p) does not precede (v_q, j_q) . From $i_p < i_q$, it follows that $v_q \rightsquigarrow v_p$, which implies that (v_q, j_q) does not precede (v_p, j_p) . Thus, Z is independent.

Conversely, assume that Z is independent. For any indices p and q , $1 \leq p < q \leq l$, it follows from $(v_p, j_p) \not< (v_q, j_q)$, $(v_q, j_q) \not< (v_p, j_p)$, $G[v_p] = B[j_p]$ and $G[v_q] = B[j_q]$ that, if $j_p = j_q$, then $v_p \not\rightsquigarrow v_q$, $v_q \not\rightsquigarrow v_p$, and $G[v_p] = G[v_q]$ hold, which contradicts the assumption that G is symbol-wise totally ordered. Hence $j_1 < j_2 < \dots < j_l$ holds. Let V_0 be an empty set, and for any index q , $1 \leq q \leq l$, let V_q be the set obtained from V_{q-1} by adding all vertices v of G such that $v \rightsquigarrow v_q$. For any index p , $1 \leq p < q$, it follows from $(v_q, j_q) \not< (v_p, j_p)$ and $j_p < j_q$ that $v_q \not\rightsquigarrow v_p$. Hence, v_q must belong to $V_q - V_{q-1}$. Let V_{l+1} be the set of all vertices of G . Then, $V_1 - V_0, V_2 - V_1, \dots, V_{l+1} - V_l$ partition the set of all vertices of G , except for the fact that the last set may be empty. For any index q , $1 \leq q \leq l + 1$, it follows from the definition of V_{q-1} that, if v is a vertex in V_{q-1} , then any vertex v' such that $v' \rightsquigarrow v$ also belongs to V_{q-1} . This implies that there exists no edge from any vertex in $V_q - V_{q-1}$ to any vertex in V_{q-1} . Therefore, the concatenation $T = T_1 T_2 \dots T_{l+1}$ of any topological sort T_q of the subgraph of G induced by $V_q - V_{q-1}$ for each index q , $1 \leq q \leq l + 1$, is a topological sort of G . Since v_q belongs to $V_q - V_{q-1}$, v_q appears in T_q for any index q , $1 \leq q \leq l$. Let i_q be the index such that $T[i_q] = v_q$. It follows that $1 \leq i_1 < i_2 < \dots < i_l \leq |T|$, and $G[T[i_q]] = B[j_q]$ for any index q , $1 \leq q \leq l$. Thus, Z represents an l -length topological common subsequence of (G, B) . $\square$

Although the problem of finding the maximal independent set is NP-hard if an arbitrary relation ' $<$ ' over any two matching pairs is given [3], the problem under the special setting of the present study is polynomial-time solvable, as shown in the following section.

4. Algorithm

In this section, an $O(|G|^2|B|^2)$ -time algorithm for computing one of the longest topological common subsequences of (G, B) is proposed.

4.1. Outline of the algorithm

According to the observation in the previous section, the proposed algorithm is designed to find one of the maximal independent sets of matching pairs of (G, B) , which represents one of the longest topological common subsequences of (G, B) . The basic idea of the algorithm is to partition the set of all matching pairs into the least number of subsets such that none of the subsets has more than one element common to any independent set of matching pairs.

Let any list $z_1, z_2, \dots, z_\ell$ of matching pairs satisfying $z_1 < z_2 < \dots < z_\ell$ be called *exclusive*, because such a list and any independent set of matching pairs cannot have more than one common elements. When there exists no possibility for confusion, any list of matching pairs is often treated as the set of all matching pairs in the list. It follows from the pigeon hole principle that, if the set of all matching pairs of (G, B) can be partitioned into L exclusive lists $R_1, \dots, R_L$, then the size of any independent set is at most L , and hence, if there exist independent sets in $R_1 \times \dots \times R_L$, then any such independent set is maximal. The proposed algorithm constructs exclusive lists $R_1, \dots, R_L$ partitioning the set of all matching pairs of (G, B) such that $R_1 \times \dots \times R_L$ has at least one independent set, and then outputs one such independent set.

To perform the above task, the algorithm inductively constructs a tuple $\mathcal{R}_j = (R_{j,1}, R_{j,2}, \dots, R_{j,L_j})$ of non-empty exclusive lists such that

- (1) $R_{j,1}, R_{j,2}, \dots, R_{j,L_j}$ partition the set of all matching pairs of (G, B_j) , and
- (2) $R_{j,1} \times \dots \times R_{j,L_j}$ has at least one independent set,

for each index j , $1 \leq j \leq |B|$, in ascending order, where we recall that B_j denotes the prefix of B of length j . Note that the set of all matching pairs of (G, B_j) consists of all matching pairs of (G, B_{j-1}) , together with all matching pairs (v, j) of (G, B_j) . Note also that, for any matching pairs (u, k) of (G, B_{j-1}) and (v, j) of (G, B_j) , (u, k) does not precede (v, j) because $k \leq j - 1$. Hence, any list $R_{j,l}$ in $\mathcal{R}_j$ must consist of a prefix of zero or more matching pairs (v, j) of (G, B_j) followed by a suffix of zero or more matching pairs of (G, B_{j-1}) . Since G is symbol-wise totally ordered, there exists a unique exclusive list Q_j that consists

of all matching pairs (v, j) of (G, B_j) . On the other hand, the tuple $\mathcal{R}_{j-1}$ consists of all matching pairs of (G, B_{j-1}) . Based on this observation, Q_j is first split into $L_{j-1} + 1$ sublists $Q_{j,1}, \dots, Q_{j,L_{j-1}+1}$ (possibly including empty lists), and for each index l , $1 \leq l \leq L_{j-1}$, $R_{j,l}$ is initialized to the concatenation $Q_{j,l}$ followed by $R_{j-1,l}$. The initial $R_{j,l}$ s are then reconstructed so that the resulting $R_{j,1}, \dots, R_{j,L_{j-1}}$, together with $Q_{j,L_{j-1}+1}$, if $Q_{j,L_{j-1}+1}$ is not empty, are non-empty exclusive lists.

As described in detail later herein, Q_j is split so that there exists an independent set Z in $R_{j-1,1} \times \dots \times R_{j-1,L_{j-1}}$ such that any element in $Q_{j,L_{j-1}+1}$ does not precede any element in Z . Since all of the resulting $R_{j,1}, \dots, R_{j,L_{j-1}}$ are exclusive, it follows from the pigeon hole principle that Z remains an independent set in $R_{j,1} \times \dots \times R_{j,L_{j-1}}$. Thus, if $Q_{j,L_{j-1}+1}$ is not empty, then $\mathcal{R}_j$ is set to $(R_{j,1}, \dots, R_{j,L_{j-1}}, Q_{j,L_{j-1}+1})$, because the union of Z and any element in $Q_{j,L_{j-1}+1}$ is an independent set in $R_{j,1} \times \dots \times R_{j,L_{j-1}} \times Q_{j,L_{j-1}+1}$. Otherwise, $\mathcal{R}_j$ is set to $(R_{j,1}, \dots, R_{j,L_{j-1}})$.

4.2. Valid suffix-exchanges

The reconstruction of $R_{j,l}$ s is performed according to the observation described below. Let a matching pair z be called a *breaking element* of a list of matching pairs, if z is the second or later element of the list, and the previous element of z in the list does not precede z . Any element of a list of matching pairs is called a *non-breaking element* of the list, if it is not a breaking element of the list. Since each $R_{j,l}$ is initially set to the concatenation of two exclusive lists $Q_{j,l}$ and $R_{j-1,l}$, if the initial $R_{j,l}$ is not exclusive, then it has exactly one breaking element, which is the first element of $R_{j-1,l}$. Let the suffix of a list of matching pairs be called the z -suffix of the list, if the first element of that suffix is z . Suppose that x is a breaking element of $R_{j,p}$, y is the previous element of x in $R_{j,p}$, and z is a non-breaking element of a different list $R_{j,q}$ such that y precedes z . Then, the exchange of the x -suffix of $R_{j,p}$ and the z -suffix of $R_{j,q}$ does not change whether w is a breaking element of one of the lists $R_{j,1}, \dots, R_{j,L_{j-1}}$, for any matching pair w except for x , which may become a non-breaking element of the resulting $R_{j,q}$. Let such an exchange of suffixes be called a *valid suffix-exchange with respect to x* , and let the matching pair z in the exchange be called the *objective matching pair*. The algorithm cancels the breaking element of the initial $R_{j,l}$ by repeatedly executing valid suffix-exchanges with respect to the first element of $R_{j-1,l}$ until it becomes a non-breaking element of one of the lists $R_{j,1}, \dots, R_{j,L_{j-1}}$, for each index l , $1 \leq l \leq L_{j-1}$, such that $Q_{j,l}$ is not empty, so that $R_{j,1}, \dots, R_{j,L_{j-1}}$ eventually become exclusive.

In order to clarify how the above computation is achieved, $\mathcal{R}_{j-1}$ must be assumed to satisfy the condition described below, which makes the concept underlying the algorithm realistic. Let C_{j-1} denote the following condition: For any index l , $1 \leq l \leq L_{j-1}$, there exists at least one independent set $\hat{Z}_{j-1,l}$ in $R_{j-1,1} \times \dots \times R_{j-1,l}$ such that

- (1) $\hat{Z}_{j-1,l}$ contains the first element of $R_{j-1,l}$, and
- (2) if $l \leq L_{j-1} - 1$, then the first element of $R_{j-1,l+1}$ does not precede any element in $\hat{Z}_{j-1,l}$,

and assume that $\mathcal{R}_{j-1}$ satisfies C_{j-1} . Later herein, it is shown that there exists a unique forefront independent set $Z_{j-1,l}$ in $R_{j-1,1} \times \dots \times R_{j-1,l}$ such that no element in any independent set in $R_{j-1,1} \times \dots \times R_{j-1,l}$ precedes any element in $Z_{j-1,l}$. It follows from condition (1) of C_{j-1} that the forefront $Z_{j-1,l}$ must contain the first element of $R_{j-1,l}$. Furthermore, no element in $Z_{j-1,l}$ precedes any element in $Z_{j-1,l-1}$, because of condition (2) of C_{j-1} for $l-1$, together with the fact that the set consisting of all elements in $Z_{j-1,l}$ except for the first element of $R_{j-1,l}$ is an independent set in $R_{j-1,1} \times \dots \times R_{j-1,l-1}$. Therefore, $\mathcal{R}_{j-1}$ is sliced into $L_{j-1} + 1$ sets $\mathcal{R}_{j-1,1}, \dots, \mathcal{R}_{j-1,L_{j-1}+1}$ of matching pairs according to these $Z_{j-1,l}$ s, where $\mathcal{R}_{j-1,l}$ is defined as the set of all matching pairs of (G, B_{j-1}) that neither precede any element in $Z_{j-1,l-1}$ nor belong to $Z_{j-1,l-1}$ if $l \geq 2$, and that either precede at least one of elements in $Z_{j-1,l}$ or belong to $Z_{j-1,l}$ if $l \leq L_{j-1}$. A conceptual example of $\mathcal{R}_{j-1}$, together with $Z_{j-1,1}, \dots, Z_{j-1,L_{j-1}}$, are given in Fig. 2. Note that the first element of $R_{j-1,l}$ is not preceded by any matching pair in $Z_{j-1,l}$ or $\mathcal{R}_{j-1,l+1}, \dots, \mathcal{R}_{j-1,L_{j-1}+1}$ because it belongs to $Z_{j-1,l}$. Hence, the objective matching pair of any valid suffix-exchange with respect to the first element of $R_{j-1,l}$ must belong to one of sets $\mathcal{R}_{j-1,1}, \dots, \mathcal{R}_{j-1,l}$ but must not belong to $Z_{j-1,l}$. This implies that the last element of $Q_{j,l}$ must precede at least one of elements in $Z_{j-1,l}$, which satisfies the condition of the objective matching pair of the first valid suffix-exchange to cancel the breaking element of the initial $R_{j,l}$. Again see Fig. 2, in which the dotted arrow from the last element of each non-empty $Q_{j,l}$ to an element in $Z_{j-1,l}$ indicates that the element of $Q_{j,l}$ precedes the element in $Z_{j-1,l}$.

The proposed algorithm explicitly determines the forefront independent set $Z_{j-1,l}$, because the process of determining $Z_{j-1,l}$ provides the series of objective matching pairs of valid suffix-exchanges for canceling the breaking element of the initial $R_{j,l}$. Recall that $Z_{j-1,l}$ represents an l -length topological common subsequence of (G, B_{j-1}) . Hence, we can see that the algorithm has, in a sense, a similar flavor to the Hunt and Szymanski algorithm [8] for computing one of the longest common subsequences of two sequences A and B , which inductively computes the forefront (i.e., the least) index i such that the length of the longest common subsequence of A_i and B_{j-1} is l .

For each index l , $1 \leq l \leq L_{j-1}$, in ascending order, the forefront $Z_{j-1,l}$ in $R_{j-1,1} \times \dots \times R_{j-1,l}$ can be obtained by the following simple procedure. Initialize Z to the union of $Z_{j-1,l-1}$ and the first element of $R_{j-1,l}$, and then repeatedly update Z by replacing any element y in Z that precedes at least one element in Z by the next element of y in the list $R_{j-1,p}$ that contains y , until Z becomes independent. Before showing the correctness of this method, let us define some notations. For any index p , $1 \leq p \leq L_{j-1}$, and any element y in $R_{j-1,p}$, let $prev(y)$ (resp. $next(y)$) denote the previous (resp. the next) element of y in $R_{j-1,p}$ unless y is the first (resp. the last) element of $R_{j-1,p}$. In addition, let $link(y)$ denote an arbitrary element in Z preceded by y just before Z is updated to $Z - \{y\} \cup \{next(y)\}$. For example, each dotted arrow in Fig. 2 from y in a list

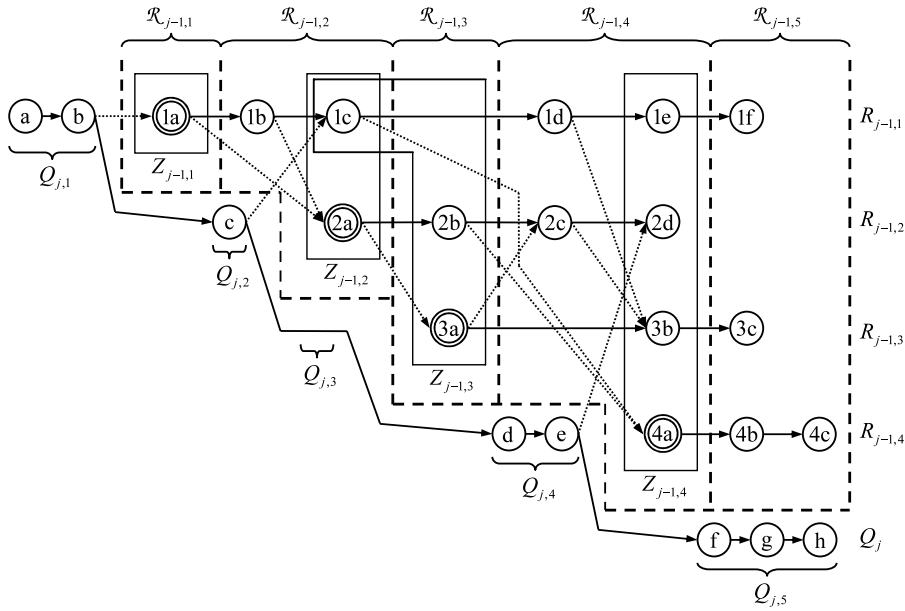


Fig. 2. Conceptual example of $\mathcal{R}_{j-1}$ and Q_j . Each circle indicates a matching pair. The circle indicating the first element of each list $R_{j-1,i}$ is doubled. Solid arrows indicate the exclusive lists $R_{j-1,1}, \dots, R_{j-1,l_{j-1}}$ and Q_j .

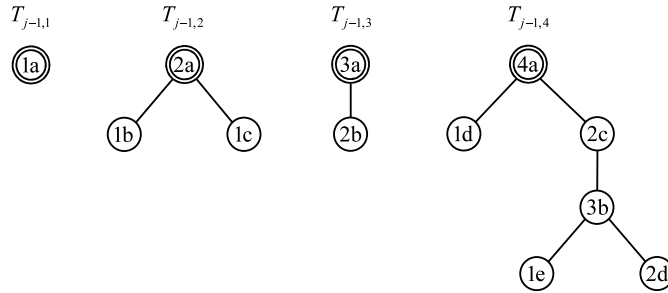


Fig. 3. Trees $T_{j-1,1}, \dots, T_{j-1,l_{j-1}}$ for $\mathcal{R}_{j-1}$ in Fig. 2.

$R_{j-1,p}$ to y' in another list $R_{j-1,q}$ indicates that $\text{link}(y) = y'$. The correctness of the method, together with the uniqueness of the forefront independent set $Z_{j-1,l}$, is verified as follows. From condition C_{j-1} , there exists at least one independent set in $R_{j-1,1} \times \dots \times R_{j-1,l}$. Let $\hat{Z}$ be an arbitrary example of such an independent set. Since the set consisting of all elements in $\hat{Z}$, except for the element common to $R_{j-1,l}$, is an independent set in $R_{j-1,1} \times \dots \times R_{j-1,l-1}$, no element in $\hat{Z}$ precedes any element in the initial Z . To show that Z eventually becomes $Z_{j-1,l}$, assume that no element in $\hat{Z}$ precedes any element in Z just before Z is updated to $Z - \{y\} \cup \{\text{next}(y)\}$. Let y belong to $R_{j-1,p}$ and let $\text{link}(y)$ belong to $R_{j-1,q}$. Note that $p \neq q$ because Z is a set in $R_{j-1,1} \times \dots \times R_{j-1,l}$ and both y and $\text{link}(y)$ belong to Z . Let z be the unique element common to $R_{j-1,p}$ and $\hat{Z}$, and let z' be the unique element common to $R_{j-1,q}$ and $\hat{Z}$. It follows from $y \leq z, y < \text{link}(y) \leq z'$ and $z \not\leq z'$ that $y < z$. Therefore, $\text{next}(y) \leq z$ holds, and hence no element in $\hat{Z}$ precedes any element in $Z - \{y\} \cup \{\text{next}(y)\}$. Recall that $\hat{Z}$ is an arbitrary independent set in $R_{j-1,1} \times \dots \times R_{j-1,l}$. Thus, whatever element y is chosen from Z during the repeated execution of such an update, Z eventually becomes a specific independent set, which is the unique forefront $Z_{j-1,l}$.

After computing $Z_{j-1,l}$, the series of objective matching pairs of valid suffix-exchanges for canceling the breaking element of the initial $R_{j,l}$ is obtained as a path of the following tree $T_{j-1,l}$, which is defined according to the process of the computation of $Z_{j-1,l}$. Initially, $T_{j-1,l}$ is set to a singleton tree, the root of which is the first element of $R_{j-1,l}$, and then a node $\text{next}(y)$ is added to $T_{j-1,l}$ as one of the children of $\text{link}(y)$ whenever Z is updated to $Z - \{y\} \cup \{\text{next}(y)\}$. Hence, the parent of any vertex z of $T_{j-1,l}$ is the matching pair $\text{link}(\text{prev}(z))$ unless z is the root of the tree. For example, the trees $T_{j-1,1}, \dots, T_{j-1,l_{j-1}}$ for $\mathcal{R}_{j-1}$ in Fig. 2 are displayed in Fig. 3. The path in $T_{j-1,l}$ from any element in $Z_{j-1,l}$ preceded by the last element of $Q_{j,l}$ to the child of the first element of $R_{j-1,l}$, i.e., the root of $T_{j-1,l}$, represents the series of objective matching pairs for canceling the breaking element of the initial $R_{j,l}$. In order to clarify the above explanation, let $z_1, z_2, \dots, z_s$ be a series of matching pairs such that the t th node in the path is z_t for any index $t, 1 \leq t \leq s$, and assume that the t th valid suffix-exchange with respect to the first element of $R_{j-1,l}$ is performed with the objective matching pair z_t . From the definition of $T_{j-1,l}$, the set of all nodes in $T_{j-1,l}$ coincides with $\mathcal{R}_{j-1,l}$, each element of which appears in $T_{j-1,l}$ exactly once. In addition, $\mathcal{R}_{j-1,l}$ contains only the first

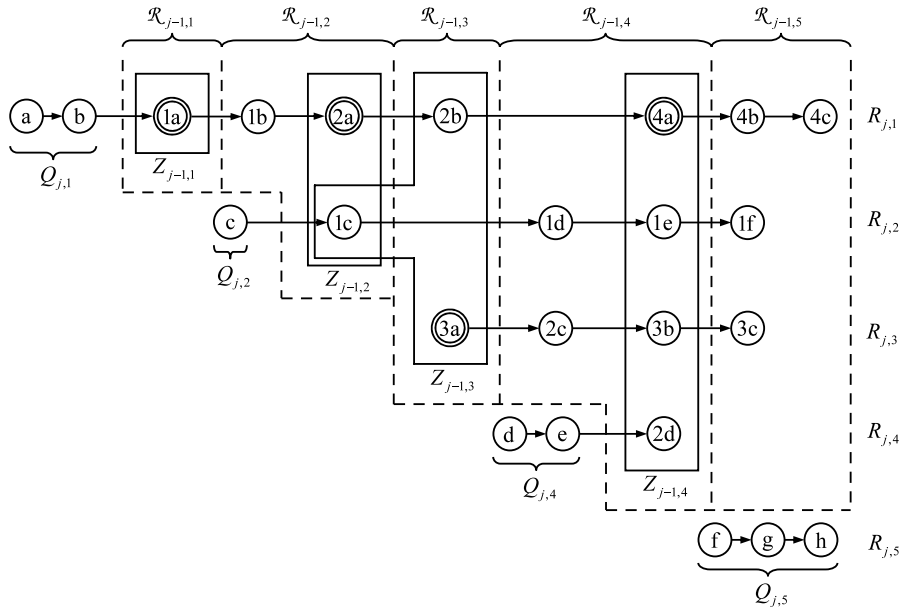


Fig. 4. The tuple $\mathcal{R}_j$ obtained from $\mathcal{R}_{j-1}$ and Q_j in Fig. 2.

element in $R_{j-1,l}$. Therefore, just before executing the t th valid suffix-exchange, if $t \geq 2$ and the first element x of $R_{j-1,l}$ belongs to $R_{j,p}$, then the previous element of x in $R_{j,p}$ is the matching pair $prev(z_{t-1})$, and hence, $z_t (= link(prev(z_{t-1})))$ does not belong to $R_{j,p}$. This implies that the suffix-change is successfully executed. Thus, after executing these s successive valid suffix-exchanges, the first element of $R_{j-1,l}$ becomes a non-breaking element of one of the resulting lists $R_{j,1}, \dots, R_{j,l}$, which means that the breaking element of the initial $R_{j,l}$ is canceled. Note that the order of execution of canceling the breaking elements of the initial lists $R_{j,1}, \dots, R_{j,l_{j-1}}$ does not affect the resulting $\mathcal{R}_j$. Fig. 4 shows $\mathcal{R}_j$ obtained from $\mathcal{R}_{j-1}$ and Q_j in Fig. 2.

What remains is to determine how to split Q_j into $Q_{j,1}, \dots, Q_{j,l_{j-1}+1}$ so that $\mathcal{R}_j$ satisfies C_l . Recall that C_l denotes the condition that, for any index l , $1 \leq l \leq L_j$, there exists at least one independent set $\hat{Z}_{j,l}$ in $R_{j,1} \times \dots \times R_{j,l}$ such that

- (1) $\hat{Z}_{j,l}$ contains the first element of $R_{j,l}$, and
- (2) if $l \leq L_j - 1$, then the first element of $R_{j,l+1}$ does not precede any element in $\hat{Z}_{j,l}$,

which guarantees the subsequent computation of $\mathcal{R}_{j+1}$ performed according to $\mathcal{R}_j$. In a similar manner to slicing $\mathcal{R}_{j-1}$ into $\mathcal{R}_{j-1,l}$ s, for any index l , $1 \leq l \leq L_{j-1} + 1$, $Q_{j,l}$ is set to the sublist of Q_j consisting of all elements that do not precede any element in $Z_{j-1,l-1}$ if $l \geq 2$, and that precede at least one of the elements in $Z_{j-1,l}$ if $l \leq L_{j-1}$. It follows from the definition of $Z_{j-1,l}$ s that the concatenation $Q_{j,1}Q_{j,2} \dots Q_{j,L_{j-1}+1}$ is identical to Q_j . Recall that, for any index l , $1 \leq l \leq L_{j-1}$, all of the objective matching pairs of valid suffix-exchanges for canceling the breaking element of the initial $R_{j,l}$ are chosen from $\mathcal{R}_{j-1,l}$. This implies that, after constructing $\mathcal{R}_j$, the independent set $Z_{j-1,l-1}$ still belongs to $R_{j,1} \times \dots \times R_{j,l-1}$. Thus, for any index l , $1 \leq l \leq L_j$, if $Q_{j,l}$ is not empty, then the union of $Z_{j-1,l-1}$ and the first element of $Q_{j,l}$ satisfies condition (1) of C_j because no element in $Z_{j-1,l-1}$ precedes the first element of $Q_{j,l}$, and vice versa. Otherwise, $Z_{j-1,l}$ satisfies condition (1) because the initial $R_{j,l}$ has no breaking element, and hence the first element of $R_{j,l}$ never changes. In addition, if $l \leq L_j - 1$, then condition (2) of C_j for l also holds because of either the definition of $Q_{j,l+1}$ or condition (2) of C_{j-1} for $l + 1$.

After constructing $\mathcal{R}_{|B|} = (R_{|B|,1}, \dots, R_{|B|,L_{|B|}})$ and $Z_{|B|,1}, \dots, Z_{|B|,L_{|B|}}$ by the method described above, the algorithm outputs the set $Z_{|B|,L_{|B|}}$ of matching pairs, which represents one of the longest topological common subsequences of (G, B) , because $R_{|B|,1}, \dots, R_{|B|,L_{|B|}}$ are exclusive lists that partition the set of all matching pairs of (G, B) , and $Z_{|B|,L_{|B|}}$ is an independent set that belongs to $R_{|B|,1} \times \dots \times R_{|B|,L_{|B|}}$.

4.3. Description of the algorithm

The description of the entire algorithm is given in Fig. 5. In the description, subscripts $j - 1$ of L_{j-1} , $R_{j-1,l}$ and $Z_{j-1,l}$ as well as subscripts j of L_j , $Q_{j,l}$, $R_{j,l}$ and $Z_{j,l}$ are omitted.

4.4. Running time analysis

As a preprocess, the algorithm constructs a binary matrix, the (u, v) th element of which for any vertices u and v of G indicates whether $u \rightsquigarrow v$, so that, for any matching pairs y and z of (G, B) , whether y precedes z can be examined in constant

- (1) Initialize L to 0;
- (2) For each index j , $1 \leq j \leq |B|$, in ascending order,
 - (a) Construct an exclusive list Q of all matching pairs (v, j) of (G, B_j) ;
 - (b) Split Q into $L+1$ sublists $Q_1, Q_2, \dots, Q_{L+1}$ (possibly including empty lists) so that Q_l consists of all elements in Q that do not precede any element in Z_{l-1} if $l \geq 2$, and that precede at least one of the elements in Z_l if $l \leq L$;
 - (c) For each index l , $1 \leq l \leq L$, if Q_l is not empty, then
 - (i) Let x be the first element of R_l ;
 - (ii) Prepend Q_l to R_l ;
 - (iii) Find an arbitrary element z in Z_l that is preceded by the last element of Q_l ;
 - (iv) While $\text{link}(\text{prev}(z))$ is not the matching pair x ,
 - (A) Exchange the x -suffix of R_p that contains x and the z -suffix of R_q that contains z ;
 - (B) Let z be the matching pair $\text{link}(\text{prev}(z))$;
 - (d) If Q_{L+1} is not empty, let $R_{L+1} = Q_{L+1}$, and then increase L by one;
 - (e) For each index l , $1 \leq l \leq L$, in ascending order,
 - (i) Let Z_l be the union of Z_{l-1} and the first element of R_l ;
 - (ii) While Z_l has elements y and z such that y precedes z ,
 - (A) Choose such arbitrary y and z in Z_l ;
 - (B) Set $\text{link}(y)$ to z ;
 - (C) Replace y in Z_l by the matching pair $\text{next}(y)$;
- (3) Output Z_L .

Fig. 5. The proposed algorithm.

time. This preprocess is performed in $O(|G|^2)$ time by a breadth-first search starting at each vertex u of G to find all vertices v of G such that $u \rightsquigarrow v$.

Let us estimate the running time of the j th iteration of statement 2. Let the list Q_j be $(v_1, j), (v_2, j), \dots, (v_s, j)$, where s is the number of vertices v of G such that $G[v] = B[j]$. Since $v_1 \rightsquigarrow v_2 \rightsquigarrow \dots \rightsquigarrow v_s$ holds from the condition that G is symbol-wise totally ordered, Q_j can be obtained by scanning all vertices of G in $O(|G|)$ time to find the first vertex v_1 , and then by executing a breadth-first search starting at v_1 , also in $O(|G|)$ time, to find all of the other vertices $v_2, v_3, \dots, v_s$ successively. To execute statement 2-b, let us regard $Z_{j-1,l}$ as a list of all matching pairs in $Z_{j-1,l}$ in arbitrary order, and let Z_{j-1} be the concatenation $Z_{j-1,1}Z_{j-1,2} \dots Z_{j-1,L_{j-1}}$. For any element z in Z_{j-1} and any element (v_t, j) in Q_j , if (v_t, j) does not precede z , then none of the subsequent elements $(v_{t+1}, j), \dots, (v_s, j)$ also precede z , because Q_j is exclusive. Thus, after initializing $Q_{j,l}$ s to empty lists, and setting z to the first element of Z_{j-1} , statement 2-b is performed in time $O(|Z_{j-1}| + |Q_j|) = O(L_{j-1}^2 + |G|)$ by finding the first element z' of the z -suffix of Z_{j-1} such that (v_t, j) precedes z' , appending (v_t, j) to $Q_{j,l}$ such that z' belongs to $Z_{j-1,l}$, or to $Q_{j,L_{j-1}+1}$ if such z' does not exist, and updating z to z' , for each index t , $1 \leq t \leq s$, in ascending order. Note that all of the elements z in statement 2-c-iii are also found during this computation. Since any valid suffix-change is executed in constant time, no matching pair of (G, B_j) ever becomes the objective for more than one valid suffix-changes, and the number of matching pairs of (G, B_j) is at most $|G||B_j|$, statement 2-c is performed in $O(|G||B_j|)$ time. Statement 2-d can be executed in constant time. Since $l-1$ candidate pairs, y and z , for testing whether y precedes z newly occur at each execution of statement 2-e-ii-C, statement 2-e is performed in $O(|G||B_j| \cdot L_{j-1})$ time. Therefore, the j th iteration of statement 2 can be executed in time $O(|G|) + O(L_{j-1}^2 + |G|) + O(|G||B_j|) + O(1) + O(|G||B_j| \cdot L_{j-1}) = O(|G|^2|B|)$ because $L_{j-1} = O(|G|)$ and $|B_j| = O(|B|)$.

Thus, the algorithm runs in time $O(|G|^2) + |B| \cdot O(|G|^2|B|) = O(|G|^2|B|^2)$.

5. Concluding remarks

This article considers a new problem of finding one of the longest topological common subsequences of a symbol-wise totally ordered dag G and a sequence B . An $O(|G|^2|B|^2)$ -time algorithm has been proposed for this problem, where $|G|$ is the number of vertices of G and $|B|$ is the length of B . It is easy to verify that this algorithm also works well in a slightly generalized setting of the problem, where vertices of G and elements of B are not necessarily labeled with symbols, but for any vertex v of G and any element b of B , whether v matches b can be tested in constant time, and for any element b of B , there exists a path of G in which all vertices that match b appear, possibly along with vertices that do not match b . Hence, in the application of the algorithm to solving the problem of aligning an irredundant multiple alignment with an additional sequence mentioned in Section 1, the condition of the multiple alignment may be relaxed, so that, for example, each column can contain two or more similar symbols under some appropriate criterion of the similarity between symbols.

It remains unclear as to whether the problem of finding one of the longest topological common subsequences of two arbitrary symbol-wise totally ordered dags G and H over Σ is solvable in polynomial time in $|G|$, $|H|$, and $|\Sigma|$. This problem has an application to merging two multiple alignments in the progressive alignment method widely employed by practical computer software for a multiple sequence alignment such as Clustal-W [13]. Recall that the algorithm proposed in this

article is similar to the Hunt and Szymanski method [8]. As one approach to the above open problem, it might be useful to seek an algorithm that is similar to the method of Needleman and Wunsch [10], which inductively computes the similarity between the prefixes of two input sequences. In the problem of finding one of the longest possible common subsequences of any sequence specified by a path of G and a sequence B , the subgraph of G induced by all paths terminated by each edge e of G is used as a prefix of G to design a Needleman–Wunsch type algorithm that runs in $O(|G|^2|B|)$ time [11,6]. Unfortunately, however, such prefixes of G do not immediately work well for the problem considered in the present article.

Acknowledgments

The author would like to thank the anonymous referees for their useful comments.

References

- [1] A. Apostolico, Improving the worst-case performance of the Hunt–Szymanski strategy for the longest common subsequences of two strings, *Inform. Process. Lett.* 23 (1986) 63–69.
- [2] C.E.R. Alves, E.N. Cáceres, S.W. Song, An all-substrings common subsequence algorithm, *Electronic Notes in Discrete Mathematics* 19 (2005) 133–139.
- [3] T.H. Cormen, C.E. Leiserson, R.L. Rivest, *Introduction to Algorithms*, MIT Press, Cambridge, MA, 1990.
- [4] F.Y.L. Chin, A. De Santis, A.L. Ferrara, N.L. Ho, S.K. Kim, A simple algorithm for the constrained sequence problems, *Inform. Process. Lett.* 90 (2004) 175–179.
- [5] J.Y. Guo, F.K. Hwang, An almost-linear time and linear space algorithm for the longest common subsequence problem, *Inform. Process. Lett.* 94 (2005) 131–135.
- [6] G. Gusfield, *Algorithms on Strings, Trees, and Sequences*, Cambridge University Press, New York, NY, 1997.
- [7] D.S. Hirschberg, A linear space algorithm for computing maximal common subsequences, *Comm. ACM* 18 (6) (1975) 341–343.
- [8] J.W. Hunt, T.G. Szymanski, A fast algorithm for computing longest common subsequences, *Comm. ACM* 20 (5) (1977) 350–353.
- [9] J. Meidanis, J.C. Setubal, Multiple alignment of biological sequences with gap flexibility, in: *Lecture Notes in Comput. Sci.*, vol. 911, Springer-Verlag, 1995, pp. 411–426.
- [10] S.B. Needleman, C.D. Wunsch, A general method applicable to the search for similarities in the amino acid sequence of two proteins, *J. Mol. Biol.* 48 (1970) 443–453.
- [11] M. Peltola, H. Soderlund, E. Ukkonen, Algorithms for the search of amino acid patterns in nucleic acid sequences, *Nucl. Acids. Res.* 14 (1986) 99–107.
- [12] A. Tiskin, Semi-local longest common subsequences in subquadratic time, *J. Discrete Algorithms* 6 (2008) 570–581.
- [13] J.D. Thompson, D.G. Higgins, T.J. Gibson, CLUSTAL W: Improving the sensitivity of progressive multiple sequence alignment through sequence weighting, position-specific gap penalties and weight matrix choice, *Nucl. Acids Res.* 22 (1994) 4673–4680.